Java Multi-Threading

Agenda

- Introduction
- Exploring Threads
- Sharing Resources
- Inter-Thread Communication
- Deadlock
- Advanced Issues
- Assignment

Introduction

- Definition
- Motivation
- Process
- Java Thread Memory Model
- Threads compared with Processes

Definition

- Objects provide a way to divide a program into independent sections. Often, you also need to turn a program into separate, independently running **subtasks**.

  Each of these independent subtasks is called a *thread*.

- Thread is a statically ordered sequence of instructions.

Motivation

- Resource utilization Programs sometimes have to wait for external operations such as input or output, and while waiting can do no useful work. It is more efficient to use that wait time to let another program run.
- Fairness Multiple users and programs may have equal claims on the machine's resources. It is preferable to let them share the computer via finer-grained time slicing than to let one program run to completion and then start another.
- Convenience Program with multiple tasks.

Process

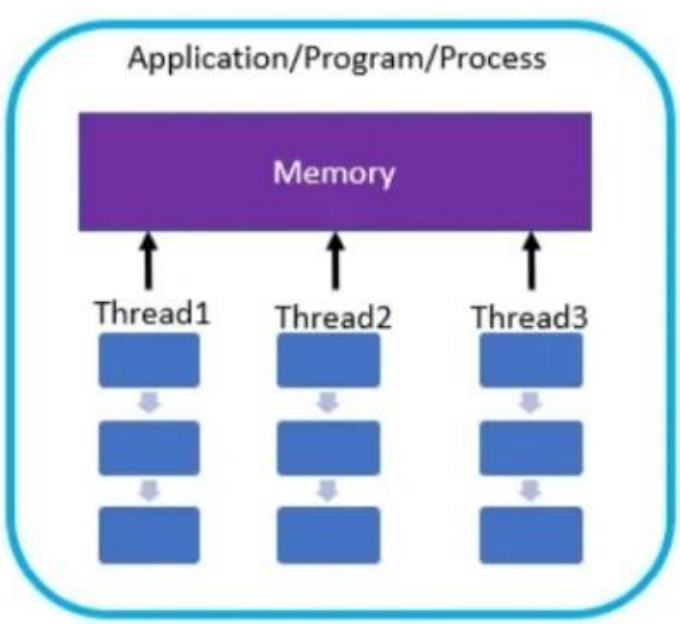
- A *process* is a self-contained running program with its own address space.
- A *multitasking* operating system is capable of running more than one process (program) at a time.

Java Thread Memory Model

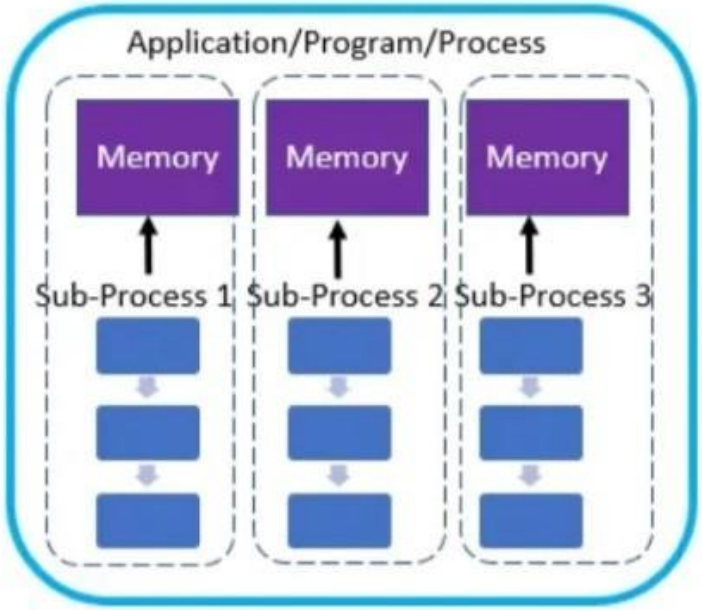
- Memory that can be shared between threads is called *shared memory* or *heap memory*. All instance fields, static fields and array elements are stored in heap memory.
- Local variables, method parameters and cached exception parameters are never shared between threads and stored in local stack and registers.

Threads compared with Processes

- Processes are typically independent, while threads exist as subsets of a process
- Processes carry considerable state information, whereas multiple threads within a process share state as well as memory and other resources
- Processes have separate address spaces, whereas threads share their address space
- Processes interact only through system-provided inter-process communication mechanisms.
- Context switching between threads in the same process is typically faster than context switching between processes.



Multi-Threading



Multi-processing

Java Threads & Synchronization

- Introduction
- Exploring Threads
- Sharing Resources
- Inter-Thread Communication
- Deadlock
- Advanced Issues
- Assignment

Exploring Threads

- Thread Life Cycle
- Thread Creation
- Priority
- Joining
- Yielding
- Sleeping
- Interrupting
- Daemon Thread

Thread Life Cycle

Threads exist in several states:

1)ready to run
2)running
3)a running thread can be suspended
4)a suspended thread can be resumed
5)a thread can be blocked when waiting for a resource
6)a thread can be terminated

Once terminated, a thread cannot be resumed.

Thread Life Cycle

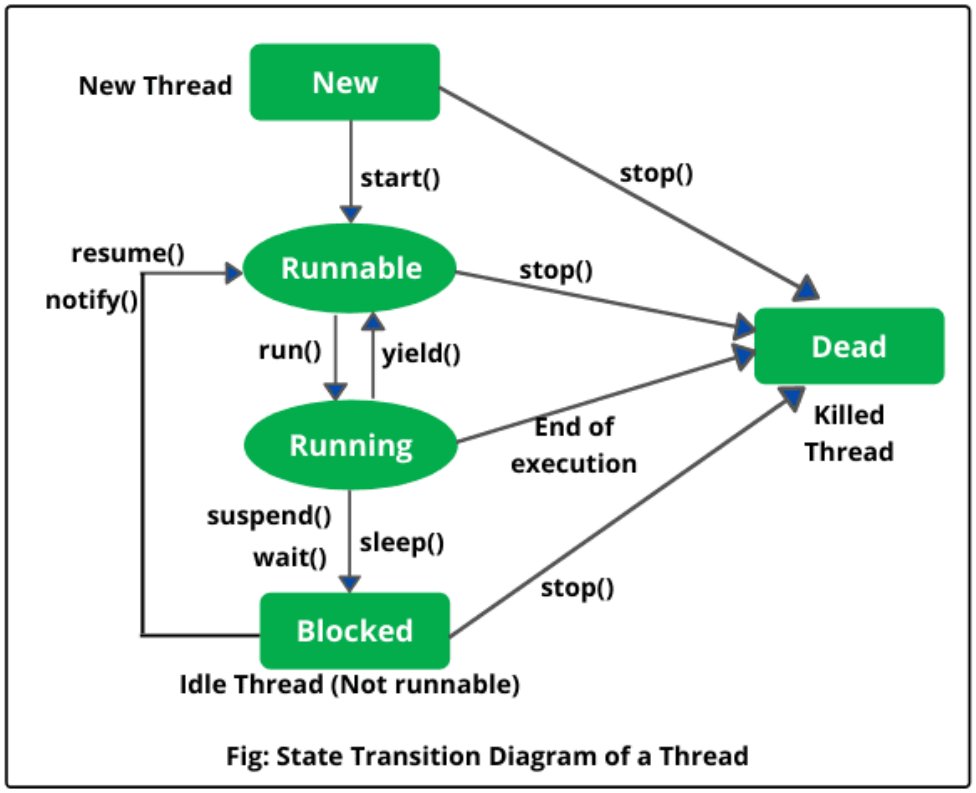


Fig: State Transition Diagram of a Thread

- New state: The thread is considered not alive.
- Runnable (Ready-to-run) state: A thread start its life. On this state a thread is waiting for a turn on the processor.
- Running state: the thread is currently executing
- Dead state: its run() method completes.
- Blocked: is waiting the resources that are hold by another thread.

Thread Creation

Two ways:

- Extending the java.lang.Thread Class
- Implementing the java.lang.Runnable Interface

Which to choose:

- If you extend the **Thread** Class, that means that subclass cannot extend any other Class, but if you implement **Runnable** interface then you can do this.
- And the class implementing the **Runnable** interface can avoid the full overhead of **Thread** class which can be excessive.
- Else use Thread

Thread Methods

- Start: a thread by calling start its run method
- Sleep: suspend a thread fora period of time
- Run: entry-point for a thread
- Join: wait for a thread to terminate
- isAlive: determine if a thread is still running
- getPriority: obtain a thread’s priority
- getName:obtain a thread’s name

Priority

- The *priority* of a thread tells the scheduler how important this thread is.
- The thread scheduler can use the thread **priorities** to determine the execution schedule of threads.
- Priorities are integer values
  - Thread.MIN_PRIORITY: 1
  - Thread.MAX_PRIORITY: 10
  - Thread.NORM_PRIORITY: 5

Joining

- One thread may call **join( )** on another thread to wait for the second thread to complete before proceeding.

Yielding

- Causes the currently executing thread to pause and allow other threads to execute.
- This hint (and it is a hint—there's no guarantee your implementation will listen to it) takes the form of the **yield()** method.
- In general, **yield()** is useful only in rare situations and you can't rely on it to do any serious tuning of your application

Sleeping

- Causes the currently executing thread to pause for a given number of milliseconds.
- When you call **sleep( )**, it must be placed inside a **try** block because it's possible for **sleep( )** to be interrupted before it times out.
- It just stops the execution of the thread for a while.
- There is no guaranty that thread will resume the execution after the given number of milliseconds.
- Not to use in real-time application.

Interrupting

- Interruption is a mechanism whereby a thread that is waiting (or sleeping) can be made to prematurely stop waiting.

- In general, InterruptedException is thrown when *another* thread interrupts the thread calling the blocking method. The other thread interrupts the blocking/sleeping thread by calling interrupt() on it.

Java Threads & Synchronization

- Introduction
- Exploring Threads
- Sharing Resources
- Inter-Thread Communication
- Deadlock
- Advanced Issues
- Assignment

Sharing Resources

- Improperly accessing resources
- Colliding over resources
- Resolving shared resource conflict
  - Semaphore
  - Mutex (Mutual Exclusion)
  - Synchronization
  - Atomic Operations
  - Volatile Variable
- Critical sections

Improperly accessing resources

- Consider the example where one task generates only even numbers.
- Scenario 1: Single Threaded Program
- Scenario 2: Multi Threaded Program

- Problem is not that the object goes through a state that violates invariance, but that methods can be called by threads while the object is in that intermediate unstable state.

Colliding over resources

- If one thread tries to read the data and other thread tries to update the same data, it leads to inconsistent state.
- A race condition occurs when the order of execution of two or more threads may affect some variable or outcome in the program.
- Race conditions can be considered harmless provided end result is correct. Otherwise needs to be handled.

Resolving shared resource conflict

- Solution: Serialize access to shared resources
- Semaphore: Semaphore is an object containing a value and two operations and used for communication between threads.
- Java has built-in support to prevent collisions over resources in the form of the **synchronized** keyword.
- It works much like the Semaphore.

Resolving shared resource conflict…

- Mutex: A mechanism in which a piece of code is running at a time by means of a lock (also called Monitor or Lock).

  Locks in Java are reentrant. Reentrancy means that locks are acquired on a per-thread basis rather than per-invocation basis.

- Synchronization: When one object pass a message to another object then both objects are in synchronized state. Synchronization needs if multiple objects passes the message to specific object.

- Atomic Operation: An *atomic operation* is one that cannot be interrupted by the thread scheduler.

- Volatile Variable: Every time the variable is used it must be read from main memory. Similarly, every time the variable is written, the value must be stored in main memory.

Synchronization (Cont…)

- Only methods (or blocks) can be synchronized, Classes and variable cannot be synchronized.
- If two threads wants to execute a synchronized method in a class, and both threads are using the same instance of the class to invoke the method then only one thread can execute the method at a time.
- If you need to synchronize one method in a class, synchronize all of them, but this is not necessary.
- You can synchronize a block of code rather than a method.
- Constructors cannot be synchronized. Code inside the constructors can be synchronized.
- Rule zero of concurrent programming: never make any assumptions.

Threads: Synchronization

- Multi-threading introduces asynchronous behavior to a program.
- How to ensure synchronous behavior when we need it?
- For instance, how to prevent two threads from simultaneously writing and reading the same object?
- Java implementation of monitors:
- 1) classes can define so-called synchronized methods
- 2) each object has its own implicit monitor that is automatically entered when one of the object's synchronized methods is called
- 3)once a thread is inside a synchronized method, no other thread can call any other synchronized method on the same object



- Language keyword: synchronized
- Takes out a monitor lock on an object
  - Exclusive lock for that thread
- If lock is currently unavailable, thread will block



- Protects access to code, not to data
  - Make data members private
  - Synchronize accessor methods
- Puts a “force field” around the locked object so no other threads can enter
  - Actually, it only blocks access to other synchronizing threads

DaemonThreads

- Any Java thread can be a daemon thread. Daemon threads are service providers for other threads running in the same process as the daemon thread.
- The run() method for a daemon thread is typically an infinite loop that waits for a service request. When the only remaining threads in a process are daemon threads, the interpreter exits. This makes sense because when only daemon threads remain, there is no other thread for which a daemon thread can provide a service.
- To specify that a thread is a daemon thread, call the setDaemon method with the argument true. To determine if a thread is a daemon thread, use the accessor method isDaemon.

Critical Sections

- Piece of code that must be executed by one thread at a time
- Must have solution that guarantees:
  - Mutual exclusion (correctness)
  - Absence of deadlock/unnecessary delay (no hang up)
  - Randomly entry (fairness)
- This is also called a *synchronized block*.

Java Threads & Synchronization

- Introduction
- Exploring Threads
- Sharing Resources
- Inter-Thread Communication
- Deadlock
- Other Stuff
- Assignment

Deadlock

- In general, want to be careful about performing any operations that might take a long time while holding a lock.

- It is possible for one thread to get stuck waiting for another thread, which in turn waits for another thread, etc., until the chain leads back to a thread waiting on the first one.

Deadlock (Cont…)

- Example 1

```
Thread1() {
    synchronized(a) {
        synchronized(b) {
            …
        }
    }
}
```

```
Thread2() {
    synchronized(b) {
        synchronized(a) {
            …
        }
    }
}
```

- // Thread1 holds lock for a, waits for b
- // Thread2 holds lock for b, waits for a

Deadlock (Cont…)

- Example 2

```
void moveMoney (Account a, Account b, int amount) {
     Synchronized (a) {
         synchronized (b) {
           a.debit (amount);
           b.credit (amount);
         }
    }
}
```

- Thread1() { moveMoney(a,b,10); }
  // holds lock for a, waits for b
- Thread2() { moveMoney(b,a,100); }
  // holds lock for b, waits for a

Thread Group

- A *Thread Group* holds a collection of threads.
- Threads in a thread group can be dealt with as a group.
    - May want to **`interrupt`** all threads in a group

"Thread groups are best viewed as an unsuccessful experiment, and you may simply ignore their existence."

Joshua Bloch

Software Architect

Oracle (Sun Microsystems)

Creating a Thread Explicitly in a Group

- A thread is a permanent member of whatever thread group it joins when its created--you cannot move a thread to a new group after the thread has been created. Thus, if you wish to put your new thread in a thread group other than the default, you must specify the thread group explicitly when you create the thread.
- The Thread class has three constructors that let you set a new thread's group: public Thread(ThreadGroup group, Runnable target) public Thread(ThreadGroup group, String name) public Thread(ThreadGroup group, Runnable target, String name)
- Each of these constructors creates a new thread, initializes it based on the Runnable and String parameters, and makes the new thread a member of the specified group.
- For example: ThreadGroup myThreadGroup = new ThreadGroup("MyGroup ofThreads"); Thread myThread = new Thread(myThreadGroup, "a thread for my group");